

INTRODUCTION

1. Product Vision

Project Name - ReserveX

A production-inspired event ticket booking platform built to study backend engineering concepts such as authentication, transactions, concurrency control, caching, asynchronous processing, observability, CI/CD, and scalability.

The frontend exists only to consume APIs. The backend is the primary focus.

2. Problem Statement

Large ticketing platforms serve thousands or even millions of users simultaneously.

When a popular event opens for booking, thousands of users may attempt to reserve the same seat within milliseconds.

Without careful system design, issues such as:

- Double booking
- Overselling
- Race conditions
- Database bottlenecks
- Lost updates
- Poor response times
- Payment inconsistencies

can occur.

This project recreates those engineering challenges on a smaller scale while following production-grade practices.

3. Business Goal

The platform allows customers to:

- Browse available events
- View seating layouts
- Hold seats temporarily
- Complete payment
- Receive booking confirmation

- View booking history

Administrators should be able to:

- Create events
- Manage venues
- Configure seating
- View bookings
- Monitor occupancy

4. Engineering Goal

The primary engineering objective is not feature completeness.

The objectives are to learn:

- Clean architecture
- Layered backend design
- REST API design
- Authentication
- Authorization
- Database modelling
- Transactions
- Optimistic and pessimistic locking
- Distributed caching
- Background jobs
- Observability
- Docker
- CI/CD
- Load testing
- Production debugging

5. Scope

Included

- User authentication
- Role-based authorization
- Event management
- Seat management
- Booking workflow
- Mock payment processing
- Refresh token rotation
- Redis caching

- Background jobs
- Structured logging
- Metrics
- Docker
- GitHub Actions
- API documentation
- Automated testing
- Load testing

Excluded

These are intentionally excluded to keep the project focused.

- Real payment gateways
- SMS integration
- Email providers
- OAuth login
- Multi-region deployment
- Kubernetes
- CDN
- Search engine
- Recommendation engine
- Real notification services
- Microservices

Where appropriate, interfaces will be designed so these can be added later without major refactoring.

6. Primary Users

Customer

Can:

- Register
- Login
- Browse events
- View seats
- Book seats
- Pay
- View bookings
- Logout

Administrator

Can:

- Create events
- Update events
- Delete events
- Configure seats
- View all bookings
- View statistics

7. Core Business Rules

These rules govern the entire system.

1. Every seat belongs to exactly one event.
2. A seat can only be in one state at a time.
3. Seat states are:

AVAILABLE
↓
HELD
↓
BOOKED

4. A held seat expires automatically if payment is not completed.
5. Only one successful booking may exist for a seat.
6. Payments can succeed or fail.
7. Failed payments must release the seat.
8. Refresh tokens must be rotatable and revocable.
9. Every important operation should be logged.
10. Every protected endpoint requires authentication.
11. Administrative operations require an administrator role.

8. Success Criteria

The project is considered successful when it demonstrates:

Functional

- Complete booking workflow
- No duplicate bookings
- Correct authentication flow
- Proper authorization
- Stable REST APIs

Engineering

- Production-style project structure
- Dockerized environment
- Automated testing
- CI pipeline
- Structured logs
- Metrics
- Background workers
- Redis caching
- Concurrency protection
- Load-tested booking flow

Functional Requirements (FR)

Objective

Functional Requirements define **what the system must do**.

These requirements describe the behavior expected from the application from the perspective of its users. They do **not** describe how the system will be implemented—that comes later during architecture and design.

Functional Modules

The system is divided into the following modules.

1. Authentication
2. User Management
3. Event Management
4. Seat Management
5. Booking Management
6. Payment Management
7. Background Jobs
8. Administration
9. Monitoring & Observability

Each module is broken down into functional requirements.

Module 1 — Authentication

FR-1.1 User Registration

The system shall allow a new user to create an account using:

- Name
- Email
- Password

Validation

- Email must be unique.
- Password must satisfy minimum security requirements.
- Password must never be stored in plain text.

FR-1.2 User Login

The system shall authenticate users using:

- Email
- Password

Upon successful authentication, the system shall issue:

- Access Token
- Refresh Token

FR-1.3 Refresh Session

The system shall allow users to obtain a new Access Token using a valid Refresh Token.

A successful refresh operation shall invalidate the previous Refresh Token and issue a new pair of tokens.

FR-1.4 Logout

The system shall revoke the current Refresh Token.

Future requests using the revoked token must fail.

FR-1.5 Authorization

The system shall support Role-Based Access Control (RBAC).

Supported roles:

- USER
- ADMIN

Protected resources shall only be accessible by authorized roles.

Module 2 — User Management

FR-2.1 View Profile

Authenticated users shall be able to retrieve their profile information.

FR-2.2 Booking History

Users shall be able to retrieve all bookings made under their account.

FR-2.3 Update Profile (Future Extension)

The architecture should allow profile updates in the future without major refactoring.

This feature will not be implemented in Version 1.

Module 3 — Event Management

FR-3.1 List Events

Users shall be able to retrieve all published events.

Optional future enhancements:

- Search
- Pagination
- Filtering
- Sorting

FR-3.2 View Event

Users shall be able to retrieve complete details for a specific event.

FR-3.3 Create Event

Administrators shall be able to create new events.

FR-3.4 Update Event

Administrators shall be able to modify event details.

FR-3.5 Delete Event

Administrators shall be able to remove unpublished events.

Future versions may implement soft deletes instead of permanent deletion.

Module 4 — Seat Management

FR-4.1 View Seat Layout

Users shall be able to view every seat belonging to an event.

Each seat shall expose:

- Seat Number
- Section
- Row
- Current Status

FR-4.2 Seat Status

Every seat shall always exist in one of the following states.

AVAILABLE

HELD

BOOKED

No other states are allowed in Version 1.

FR-4.3 Hold Seat

The system shall temporarily reserve a seat during the booking process.

A held seat shall become unavailable to other users.

FR-4.4 Seat Expiration

Held seats shall automatically become available again after a configurable timeout if payment is not completed.

Module 5 — Booking Management

FR-5.1 Create Booking

Authenticated users shall be able to initiate a booking for an available seat.

The system shall ensure:

- only one booking succeeds
- duplicate bookings are prevented
- booking creation is transactional

FR-5.2 View Booking

Users shall be able to retrieve booking details.

FR-5.3 Booking Status

Bookings shall exist in one of the following states.

PENDING

CONFIRMED

FAILED

CANCELLED

EXPIRED

FR-5.4 Booking Expiration

Pending bookings shall automatically expire after the configured payment timeout.

Module 6 — Payment

Version 1 intentionally uses a mock payment provider.

FR-6.1 Initiate Payment

Users shall be able to submit payment for a pending booking.

FR-6.2 Payment Success

Successful payment shall:

- confirm booking
- mark seat as BOOKED
- generate confirmation

FR-6.3 Payment Failure

Failed payment shall:

- mark payment as failed
- release held seat
- update booking status

FR-6.4 Payment Status

Supported states:

PENDING

SUCCESS

FAILED

REFUNDED

Refunds will not be implemented but the state is reserved for future expansion.

Module 7 — Background Jobs

FR-7.1 Seat Expiration Job

The system shall automatically release expired seat holds.

FR-7.2 Confirmation Job

Successful bookings shall trigger an asynchronous confirmation task.

Version 1 will simulate this by writing structured logs.

FR-7.3 Retry Failed Jobs

Temporary worker failures should be retried automatically.

Module 8 — Administration

Administrators shall be able to:

- Create events
- Modify events
- Delete events
- View bookings
- View seat occupancy
- View booking statistics

Administrative APIs shall not be accessible by regular users.

Module 9 — Monitoring & Observability

The system shall expose operational information.

This includes:

- Application health

- Request metrics
- Booking metrics
- Error metrics
- Structured logs

These features are intended for operators rather than end users.

Future Functional Extensions

The architecture should support future additions without major redesign.

Examples include:

- Email notifications
- SMS notifications
- QR code tickets
- Waiting lists
- Discount coupons
- Seat categories
- Dynamic pricing
- Multiple venues
- Multiple organizers
- Payment gateway integration
- OAuth authentication
- Event recommendations
- Search engine integration

None of these are part of Version 1.

Functional Requirement Traceability

Module	Status
Authentication	Version 1
User Management	Version 1
Event Management	Version 1
Seat Management	Version 1
Booking Management	Version 1
Payment (Mock)	Version 1
Background Jobs	Version 1
Administration	Version 1
Monitoring	Version 1
Advanced Features	Future

